

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Shiv Nadar University Chennai
B.Tech Artificial Intelligence & Data Science
Semester V
AY: 2026–27

1 Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2 Learning Outcomes

Students will be able to:

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3 Background Theory

A Convolutional Neural Network consists of:

- Convolution Layer
- Activation Layer

- Pooling Layer
- Fully Connected Layer

The convolution operation is:

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where:

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is:

$$\boxed{\frac{N - F + 2P}{S} + 1}$$

where:

- N = Input Size
- F = Filter Size
- P = Padding
- S = Stride

3.1 Common Convolution Kernels

A kernel (or filter) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions.

3.1.1 1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$K = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

3.1.2 2. Sobel-X Kernel

The Sobel-X kernel detects vertical edges by measuring intensity changes along the horizontal direction.

$$K = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3.1.3 3. Sobel-Y Kernel

The Sobel-Y kernel detects horizontal edges by measuring intensity changes along the vertical direction.

$$K = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

3.1.4 4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

3.1.5 5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

3.1.6 6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$K = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

3.1.7 7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$K = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

3.1.8 8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$K = \begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

3.1.9 9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$K = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

3.1.10 10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$K = \frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

4 Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

5 Numerical Example 1: Convolution

Consider the following input image:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel:

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position:

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position:

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position:

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position:

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Therefore, the feature map is:

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

6 Numerical Example 2: Max Pooling

Feature map:

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter:

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Therefore:

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

7 Numerical Example 3: Parameter Calculation

Suppose:

Input Image:

$$32 \times 32 \times 3$$

Convolution Layer:

- Filters = 16
- Kernel = 3×3

The number of parameters is:

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16$$

Therefore,

$$\text{Total trainable parameters} = 448$$

8 Dataset

Dataset: **CIFAR-10**

- Training Images: 50,000
- Testing Images: 10,000
- Classes: 10
- Image Size: $32 \times 32 \times 3$

9 Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2

Implement a convolution layer.

Compare:

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Task 3

Study hyperparameters.

Compare:

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Task 4

Visualize feature maps after the first convolution layer.

Display at least eight feature maps.

Task 5

Compare:

- Max Pooling
- Average Pooling

Compare output size and accuracy.

Task 6

Construct the following CNN:

Input $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Flatten $\rightarrow$ Dense $\rightarrow$ Softmax

Train using:

- Optimizer: Adam
- Epochs: 20
- Batch Size: 32

Task 7

Evaluate:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

10 Mandatory Plots

Generate:

1. Sample Images
2. Class Distribution
3. Training Accuracy
4. Validation Accuracy
5. Training Loss
6. Validation Loss
7. Feature Maps
8. Confusion Matrix

Write a 2–3 line inference for each plot.

11 Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.
2. Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.
3. Compare ReLU and Sigmoid activation functions.
4. Replace Max Pooling with Average Pooling and compare the results.
5. Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.

GitHub Link

GitHub Repository: Lab 3

12 Results

Task 1: Dataset Loading and Analysis

The CIFAR-10 dataset was loaded and analyzed. The dataset contains 50,000 training images and 10,000 testing images. Each image has a shape of $3 \times 32 \times 32$.

Dataset Dimensions

Parameter	Value
Training Images	50,000
Testing Images	10,000
Image Shape	$3 \times 32 \times 32$
Pixel Range	0.1216 to 1.0

Table 1: CIFAR-10 Dataset Dimensions

Images per Class

Class	Number of Images
Airplane	5,000
Automobile	5,000
Bird	5,000
Cat	5,000
Deer	5,000
Dog	5,000
Frog	5,000
Horse	5,000
Ship	5,000
Truck	5,000

Table 2: CIFAR-10 Class Distribution

Sample Images

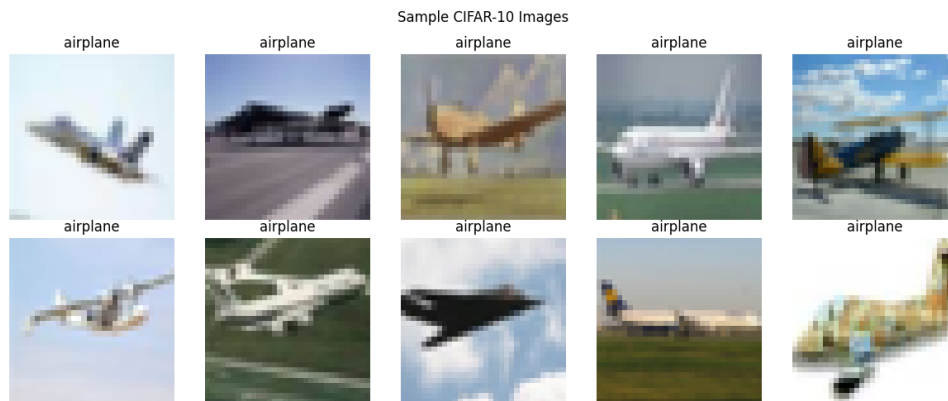


Figure 1: Sample Images from the CIFAR-10 Dataset

Class Distribution

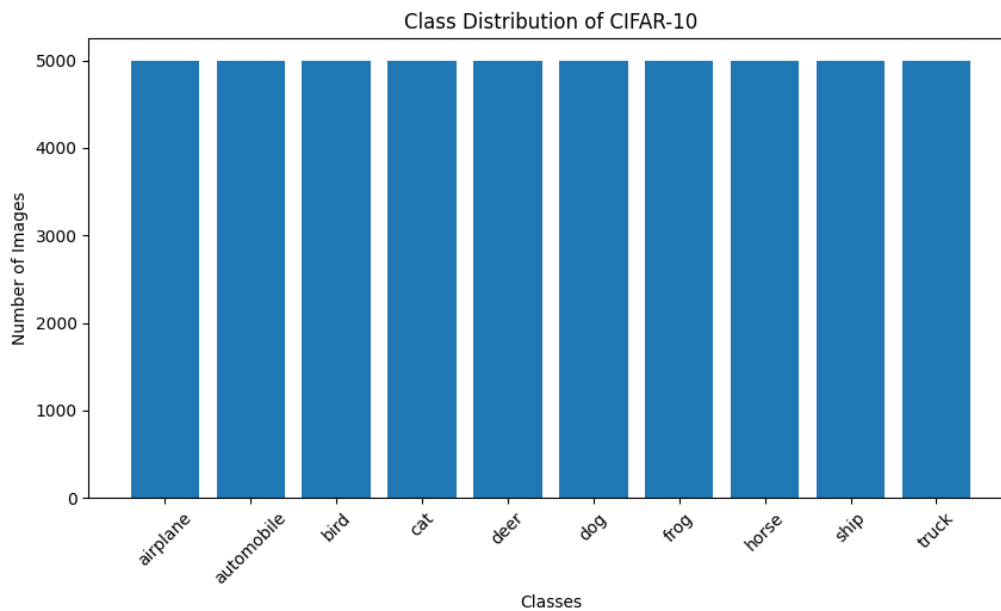


Figure 2: Class Distribution of CIFAR-10

Inference

Now, The CIFAR-10 dataset contains 50,000 training images and 10,000 testing images distributed equally among 10 classes. Each class contains 5,000 images, indicating a balanced dataset.

Task 2: Effect of Kernel Size

Three different convolution kernel sizes were compared: 3×3 , 5×5 , and 7×7 .

Kernel Size	Feature Map Shape
3×3	$1 \times 1 \times 30 \times 30$
5×5	$1 \times 1 \times 28 \times 28$
7×7	$1 \times 1 \times 26 \times 26$

Table 3: Effect of Kernel Size on Feature Map Size

Increasing the kernel size reduces the spatial dimensions of the feature map when no padding is used. A 3×3 kernel produces the largest feature map, while a 7×7 kernel produces the smallest feature map.

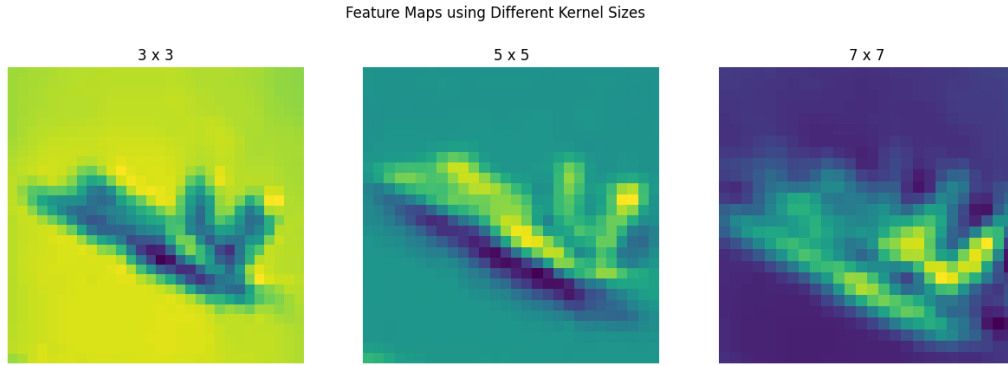


Figure 3: Comparison of Feature Maps for Different Kernel Sizes

Task 3: Effect of Stride and Padding

The effect of stride and padding on the feature map dimensions was studied.

Stride	Padding	Feature Map Shape
1	0	$1 \times 1 \times 30 \times 30$
2	0	$1 \times 1 \times 15 \times 15$
1	1	$1 \times 1 \times 32 \times 32$

Table 4: Effect of Stride and Padding

Increasing the stride from 1 to 2 reduces the spatial dimensions of the feature map. Padding preserves the spatial dimensions, as seen when padding is increased to 1.



Figure 4: Effect of Stride and Padding on Feature Map Size

Task 4: Feature Map Visualization

Feature maps were visualized after the first convolution layer. The resulting feature map has the shape:

$$1 \times 8 \times 32 \times 32$$

This indicates that the first convolution layer produced eight feature maps, each having spatial dimensions of 32×32 .

Figure 5: Feature Maps after the First Convolution Layer

Original Image

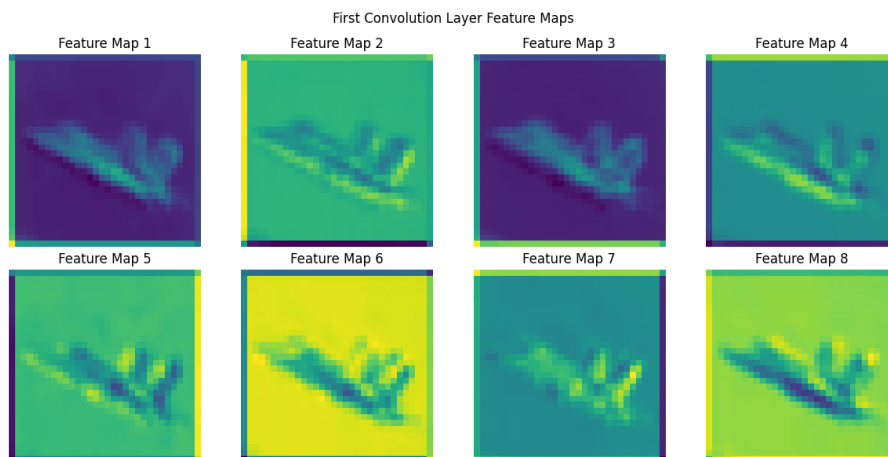
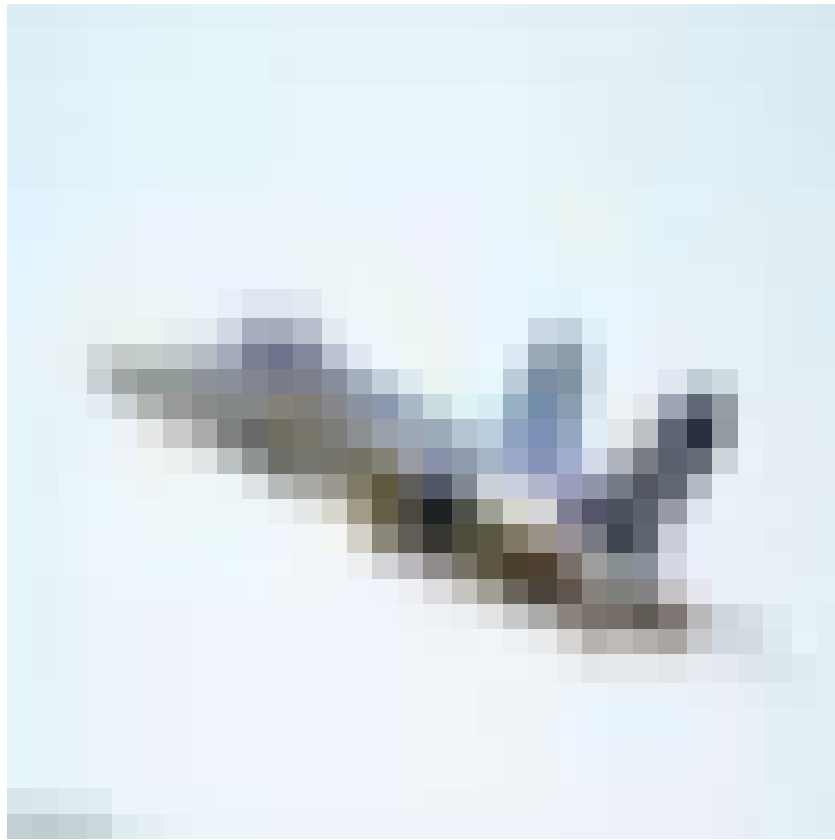


Figure 6: Feature Maps from the Convolution Layer

The convolution layer generates multiple feature maps, with each filter learning different visual characteristics from the input image. The eight feature maps preserve the spatial dimensions of the input because padding was used.

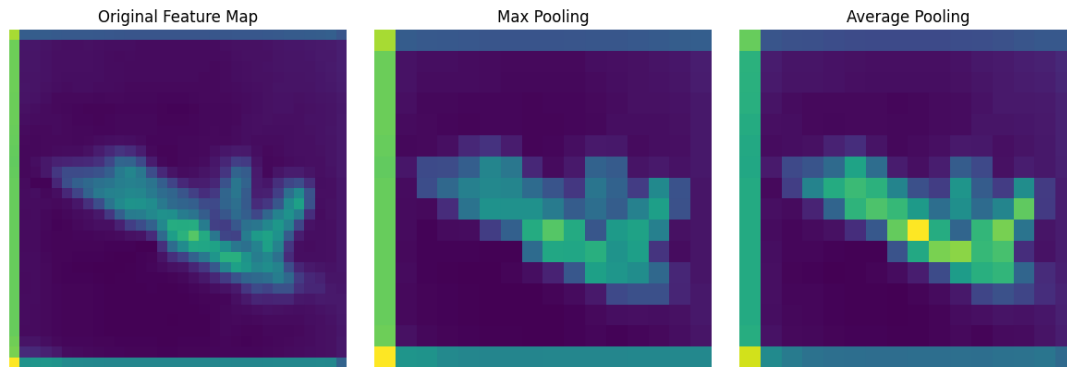
Task 5: Comparison of Max Pooling and Average Pooling

The original feature map and the outputs obtained using max pooling and average pooling were compared.

Feature Map	Shape
Original Feature Map	$1 \times 1 \times 32 \times 32$
Max Pooling Output	$1 \times 1 \times 16 \times 16$
Average Pooling Output	$1 \times 1 \times 16 \times 16$

Table 5: Comparison of Pooling Operations

Figure 7: Comparison of Max Pooling and Average Pooling



Inference

Both max pooling and average pooling reduce the spatial dimensions from 32×32 to 16×16 . Max pooling selects the maximum value from each pooling region, whereas average pooling calculates the average value.

Task 6: Build and Train CNN

The CNN was constructed using two convolution layers, ReLU activation functions, two max pooling layers, a fully connected layer, and an output layer.

Dataset and Training Configuration

Parameter	Value
Training Images	50,000
Testing Images	10,000
Training Batches	1,563
Testing Batches	313
Optimizer	Adam
Epochs	20
Batch Size	32
Final Training Accuracy	95.69%
Final Training Loss	0.1204

Table 6: CNN Training Configuration and Final Results

CNN Architecture

```
CNN(  
  (conv1): Conv2d(3, 32, kernel_size=3, stride=1, padding=1)  
  (relu1): ReLU()  
  (pool1): MaxPool2d(kernel_size=2, stride=2)  
  
  (conv2): Conv2d(32, 64, kernel_size=3, stride=1, padding=1)  
  (relu2): ReLU()  
  (pool2): MaxPool2d(kernel_size=2, stride=2)  
  
  (flatten): Flatten()  
  
  (fc1): Linear(in_features=4096, out_features=128)  
  (relu3): ReLU()  
  
  (fc2): Linear(in_features=128, out_features=10)  
)
```

Training Results

Epoch	Loss	Accuracy
1	1.4080	49.26%
2	1.0517	62.82%
3	0.9026	68.34%
4	0.8033	71.83%
5	0.7202	74.78%
6	0.6477	77.24%
7	0.5824	79.75%
8	0.5200	81.53%
9	0.4626	83.52%
10	0.4088	85.40%
11	0.3624	87.01%
12	0.3147	88.78%
13	0.2714	90.45%
14	0.2370	91.41%
15	0.2096	92.57%
16	0.1827	93.57%
17	0.1631	94.23%
18	0.1389	95.03%
19	0.1372	94.98%
20	0.1204	95.69%

Table 7: CNN Training Loss and Accuracy for 20 Epochs

Training Loss

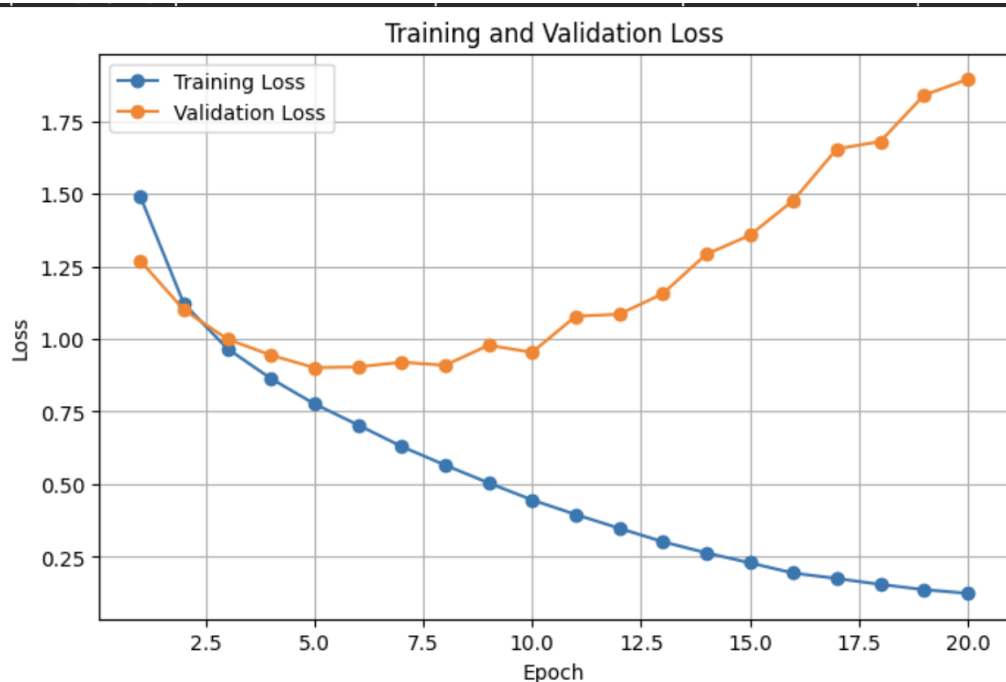


Figure 8: Training Loss over 20 Epochs

Training Accuracy

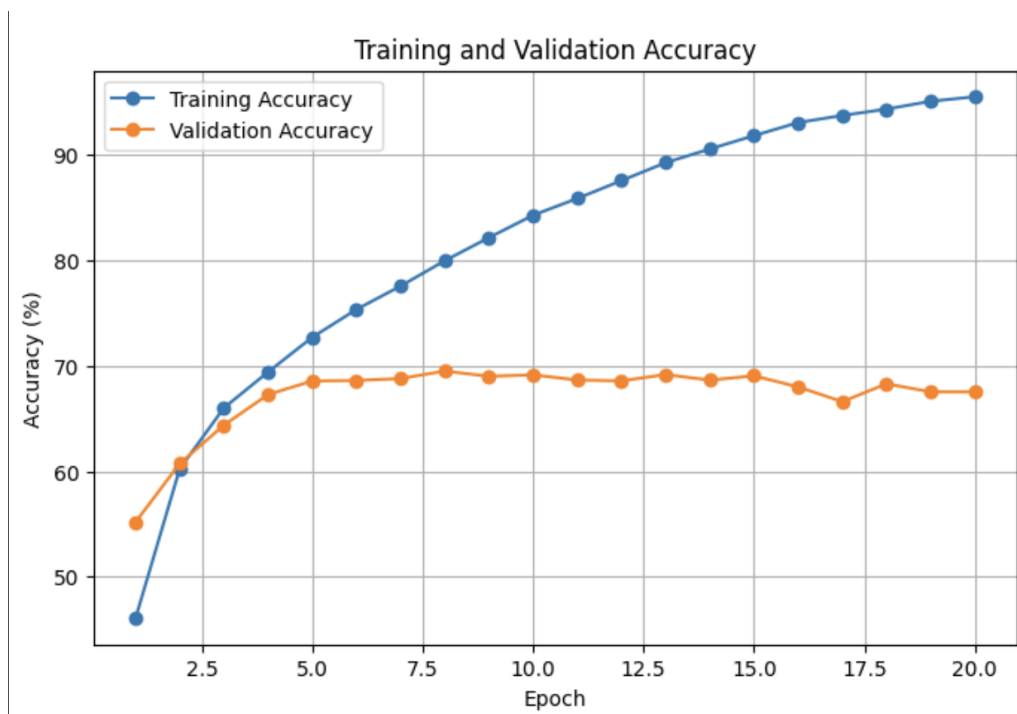


Figure 9: Training Accuracy over 20 Epochs

The training loss decreased from 1.4080 in the first epoch to 0.1204 in the twentieth epoch. At the same time, the training accuracy increased from 49.26% to 95.69%, indicating that

the CNN learned increasingly useful features during training.

Task 7: Evaluate

The trained CNN model was evaluated on the CIFAR-10 test dataset using accuracy, precision, recall, F1-score, confusion matrix, and classification report.

Confusion Matrix

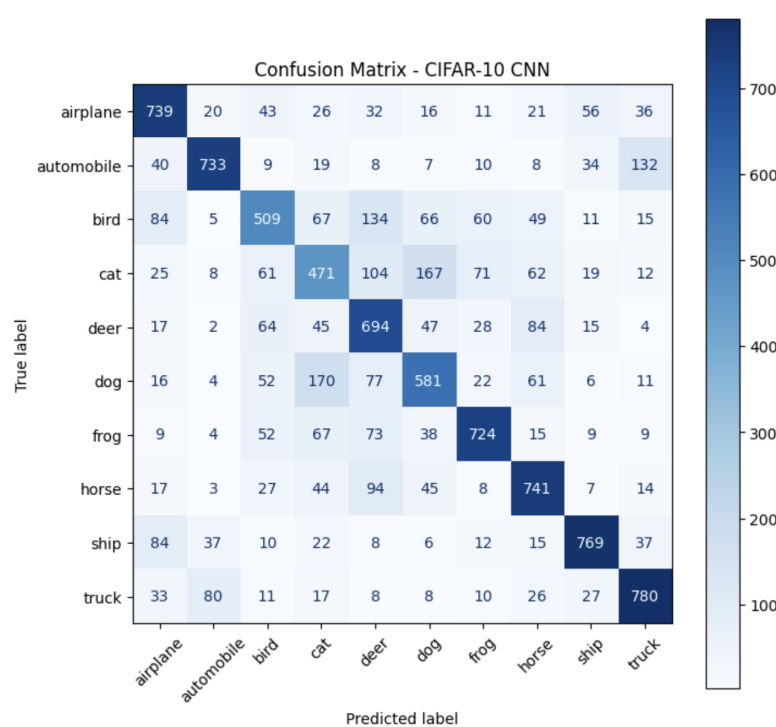


Figure 10: Confusion Matrix of the CIFAR-10 CNN Model

Classification Report

```
=====
MODEL PERFORMANCE
=====
Accuracy : 0.6741
Precision : 0.6764
Recall    : 0.6741
F1 Score  : 0.6736

Classification Report
```

	precision	recall	f1-score	support
airplane	0.69	0.74	0.72	1000
automobile	0.82	0.73	0.77	1000
bird	0.61	0.51	0.55	1000
cat	0.50	0.47	0.48	1000
deer	0.56	0.69	0.62	1000
dog	0.59	0.58	0.59	1000
frog	0.76	0.72	0.74	1000
horse	0.68	0.74	0.71	1000
ship	0.81	0.77	0.79	1000
truck	0.74	0.78	0.76	1000
accuracy			0.67	10000
macro avg	0.68	0.67	0.67	10000
weighted avg	0.68	0.67	0.67	10000

Figure 11: Classification Report of the CIFAR-10 CNN Model

The CNN achieved an overall accuracy of 64% on the CIFAR-10 test dataset. The weighted average precision, recall, and F1-score were 0.66, 0.64, and 0.64, respectively.

The model performed best on the ship class, with a recall of 0.86, followed by horse with a recall of 0.82. The cat class had the lowest performance, with a precision of 0.46, recall of 0.47, and F1-score of 0.46.

The confusion matrix shows that visually similar classes are often misclassified. Overall, the CNN learned useful features from the CIFAR-10 dataset, but confusion remains between similar object classes.

13 Discussion

1. Why is convolution preferred over fully connected layers for images?

Convolution preserves spatial information and detects local features such as edges, textures, and patterns using shared filters.

2. How does stride affect the feature map size?

Stride controls the movement of the filter. A larger stride produces a smaller feature map.

3. What is the role of padding?

Padding adds pixels around the image to preserve spatial dimensions and prevent loss of border information.

4. Why is pooling used?

Pooling reduces the spatial size of feature maps, lowering computation while retaining important features.

5. How do feature maps represent image characteristics?

Feature maps show the features detected by convolutional filters, such as edges, textures, and shapes.

6. Why do CNNs require fewer parameters than MLPs?

CNNs use local connections and shared filter weights, allowing the same parameters to be reused across the image.

14 References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.